```
        if (string.IsNullOrEmpty(HashedPassword))                          ❶
        {                                                                  ❶
            HashedPassword = _passwordHasher.HashPassword(user, Password);❶
            Result = "Password has been hashed. Please sign in.";          ❶
        }                                                                  ❶
        else                                                               ❷
        {                                                                  ❷
            Result = _passwordHasher.VerifyHashedPassword(                 ❷
            ➥ user, HashedPassword, Password).ToString();                 ❷
        }                                                                  ❷
    }
}
```

❶ If this hidden field is empty, this is the first time the form has been submitted. Hash the password.

❷ Otherwise, you compare the submitted password to the one you created previously.

Run the application, and enter a username and password. Submit the form, and once it is displayed again, enter either the same password or a different one to observe the result. If you enter the same password, you should see the word Success above the form in green. Otherwise, you should see Failed. These are two of the three values in the `PasswordVerificationResult` enum returned by the `VerifyHashedPassword` method. The third value, `SuccessRehashNeeded`, says that verification was successful, but the password needs rehashing because it was originally hashed using a deprecated algorithm. This would be the case if you were to increase the number of iterations at some stage, which you may well do to ensure your application keeps up with the latest recommendations.

This contrived example serves purely to demonstrate usage of the `PasswordHasher` independent of the Identity UI and default storage. In a real-world application, you would store the hashed password in a permanent store and then retrieve it for verification. You don't actually need to take a dependency on the Identity package at all because you can use the Key Derivation APIs directly: http://mng.bz/O6Aw.

That takes care of protecting your users' passwords, but what about other types of passwords, such as those required in database connection strings or mail server credentials? Your existing connection string doesn't include a password, but it is commonly required for databases that support user logins. In addition, your application may need to store client secrets for connecting to third-party services. Storing those in plain text within application code presents a security risk.